

CAHIER DES CHARGES

Projet d'application informatique

SAVORA

Application mobile de livraison de repas

React Native (Expo · TypeScript) · Node.js / Express · MongoDB

Équipe : Global SoftCorporation

ARCHANGE GUIMDO · JORDAN DONGMEZA · YAN SAH · WILFRED GALIMAR

Cours : Documentation technique — Institut Teccart

Enseignant : Safouen Bani

Version 3.1 — 2 août 2026

Table des matières

Note sur la version 3.1

Ce cahier des charges décrit un projet livré, non un projet à venir. Il a été rédigé après la fin du développement, en confrontant chaque exigence au code réellement écrit. La version 3.1 ajoute trois fonctionnalités livrées après la 3.0 : la gestion du catalogue depuis l'application, les notifications à chaque étape, et le déploiement du backend.

Trois corrections majeures ont été apportées par rapport à la version 2.0 :

- Périmètre — l'application couvre désormais quatre rôles (client, restaurant, livreur, administrateur) et une discussion par commande, alors que la v2.0 classait ces éléments en « Won't have ». La justification de cet élargissement figure en section 16.
- Pile mobile — le projet utilise TypeScript et Expo Router, et non JavaScript avec react-navigation. Redux a été écarté au profit de la seule Context API.
- Paiement — Stripe fonctionne en mode test côté serveur, avec un repli automatique sur une simulation locale lorsque aucune clé n'est configurée, afin que la démonstration ne dépende pas du réseau.

Deux sections ont été ajoutées : les écarts assumés par rapport à la version 2.0 (section 16) et la dette technique connue (section 17). Une exigence non tenue et documentée vaut mieux qu'une exigence tenue sur le papier seulement.

Historique des versions

Version	Date	Nature de la révision
1.0	Mai 2026	Cadrage initial. Périmètre irréaliste : quatre applications complètes pour quatre personnes en douze semaines.
2.0	Juin 2026	Recadrage sur un MVP centré sur l'application client. Ajout de la priorisation MoSCoW, des user stories, des exigences mesurables, de la conformité Loi 25, des risques et de la traçabilité.
3.0	Août 2026	Mise en conformité avec le produit livré : quatre rôles, discussion, notation, Stripe en mode test. Ajout des écarts assumés et de la dette technique.
3.1	Août 2026	Gestion du catalogue depuis l'application, notifications à chaque étape, déploiement Render et Railway. Trois lignes de dette technique soldées.

1. Contexte et justification

Cuisiner prend du temps. Étudiants, travailleurs et familles pressées veulent commander un repas rapidement, payer en ligne et suivre la livraison sans téléphoner à personne. Savora répond à ce besoin en réunissant clients, restaurants et livreurs dans une seule application mobile.

Le client (commanditaire fictif)

Le projet est réalisé dans le cadre du cours de documentation technique de l'Institut Teccart. Le commanditaire est une entreprise de restauration locale souhaitant offrir un service de commande et de livraison en ligne, inspiré des plateformes existantes mais ramené à un périmètre de démonstration réaliste.

Besoins principaux

- Commander un repas dans le restaurant de son choix et suivre la commande en temps réel.
- Personnaliser sa commande (suppléments, formats, options).
- Payer en ligne de façon sécurisée, ou à la livraison.
- Échanger avec le restaurant ou le livreur en cas de besoin.
- Évaluer son expérience après la livraison.

2. Objectifs de l'application

Objectif général

Faciliter la commande et la livraison de repas en offrant une application mobile simple, fluide et sécurisée, dont le parcours complet est démontrable de bout en bout.

Objectifs spécifiques

- Permettre à un client de parcourir un menu interactif et de personnaliser ses plats.
- Offrir un suivi en temps réel du statut de la commande, de la préparation à la livraison.
- Intégrer un paiement en ligne sécurisé (Stripe, mode test).
- Permettre la notation des restaurants après la livraison.
- Donner aux restaurants et aux livreurs les moyens de faire réellement avancer une commande.
- Livrer une documentation technique complète, continue et alignée sur le code.

3. Périmètre du projet et priorisation MoSCoW

Le périmètre a été élargi en cours de projet : construire un suivi de commande en temps réel sans acteur pour faire avancer les statuts revenait à simuler la fonctionnalité imposée plutôt qu'à la livrer. La priorisation ci-dessous reflète l'état final.

Priorité	Fonctionnalité	État	Justification
Must	Inscription / connexion (JWT, bcrypt)	Livré	Sans compte, pas de commande
Must	Menu interactif et personnalisation des plats	Livré	Fonctionnalité 1 imposée
Must	Panier et passage de commande	Livré	Cœur du parcours
Must	Suivi de commande en temps réel (Socket.IO)	Livré	Fonctionnalité 2 imposée
Must	Paieement intégré (Stripe mode test)	Livré	Fonctionnalité 3 imposée
Must	Notation des restaurants	Livré	Fonctionnalité 4 imposée
Must	Espace restaurant (changement de statut)	Livré	Rend le suivi temps réel réel
Should	Espace livreur (prise en charge, livraison)	Livré	Sans lui, aucun statut après « prête »
Should	Recherche de restaurants	Livré	Améliore l'expérience
Should	Historique des commandes	Livré	Utile, non bloquant
Could	Espace administrateur	Livré	Statistiques et modération
Could	Discussion par commande	Livré	Né du besoin « où est mon livreur ? »
Could	Carte de livraison	Partiel	Position du livreur réelle, restaurant et client fixes
Should	Gestion du menu par le restaurant	Livré	Le catalogue n'était modifiable que par script
Should	Création de restaurants par l'administration	Livré	Établissement et compte gestionnaire créés ensemble
Could	Notifications à chaque étape	Livré	Push distantes, avec repli local pour Expo Go
Could	Déploiement du backend	Livré	Configurations Render et Railway versionnées
Won't	Connexion Google, pourboires, modération des avis	Non livré	Reportés en perspectives

4. Description fonctionnelle

L'application réunit quatre espaces dans une seule base de code. Après connexion, l'utilisateur est redirigé vers l'espace correspondant à son rôle.

4.1 Espace client

Compte et authentification

- Inscription et connexion par courriel et mot de passe (8 caractères minimum).
- Profil : nom, téléphone, adresses de livraison.
- Session conservée entre deux lancements, dans le trousseau sécurisé du système.
- Réinitialisation de mot de passe par jeton à usage unique valable une heure.

Menu interactif — Fonctionnalité 1

- Liste des restaurants avec photo, type de cuisine, note moyenne, délai et frais de livraison.
- Recherche par nom ou par type de cuisine.
- Fiche restaurant avec le menu complet.
- Personnalisation des plats : suppléments et formats, avec mise à jour du prix en direct.

Panier et commande

- Ajout, retrait et modification des quantités. Deux personnalisations différentes d'un même plat forment deux lignes distinctes.
- Le panier ne contient qu'un seul restaurant à la fois.
- Calcul du sous-total, des frais de livraison et des taxes du Québec (14,975 %).

Paielement — Fonctionnalité 3

- Carte bancaire traitée par Stripe en mode test, ou par simulation locale hors ligne.
- Paiement à la livraison, encaissé à la remise du repas.
- Aucun numéro de carte n'est stocké : seule la référence de transaction l'est.

Suivi en temps réel — Fonctionnalité 2

- Sept statuts visibles, de « en attente » à « livrée ».
- Mise à jour automatique par Socket.IO, sans rafraîchissement manuel.
- Estimation du temps restant et historique horodaté des statuts.

Notation — Fonctionnalité 4

- Note de 1 à 5 étoiles et commentaire facultatif, uniquement après livraison.

- Une seule notation par commande, garantie par un index unique en base.
- Note moyenne du restaurant recalculée immédiatement et affichée sur sa fiche.

4.2 Espace restaurant

- Réception des nouvelles commandes en temps réel.
- Progression des statuts : confirmée, en préparation, prête.
- Annulation possible tant que la commande n'est pas prête.
- Gestion du menu : ajout, modification et retrait de plats, avec leurs options de personnalisation et leurs suppléments.
- Modification de la fiche de l'établissement, à l'exception de son activation qui reste une prérogative de l'administration.
- Cloisonnement strict : un compte restaurant n'accède qu'aux commandes et au menu de son établissement.

4.3 Espace livreur

- Liste en temps réel des commandes prêtes et non attribuées.
- Acceptation d'une livraison. L'attribution est atomique : si deux livreurs acceptent au même instant, un seul obtient la course.
- Progression : en route, puis livrée.
- Carte de suivi utilisant la position réelle du téléphone du livreur.

4.4 Espace administrateur

- Statistiques : utilisateurs par rôle, restaurants actifs, commandes par statut, revenus des commandes livrées.
- Gestion des utilisateurs : changement de rôle, suppression (hors son propre compte).
- Création d'un établissement et, en une seule opération, de son compte gestionnaire.
- Gestion des restaurants : modification de la fiche, activation, désactivation, suppression.
- Gestion du menu de n'importe quel établissement.
- Gestion des commandes : consultation filtrée et annulation.

4.5 Notifications

- Une notification à chaque étape franchie par la commande, de la confirmation à la livraison.
- Le restaurant est averti de l'arrivée d'une nouvelle commande.

- Deux canaux complémentaires : notifications poussées distantes, et notification locale déclenchée par le canal temps réel.
- Aucune notification au moment où le client passe commande : il vient de la passer.

4.6 Discussion par commande

- Une discussion est rattachée à chaque commande.
- Participants autorisés : le client concerné, le restaurant, le livreur assigné et l'administration.
- Messages en temps réel, limités à 1000 caractères.

5. User stories et critères d'acceptation

Chaque fonctionnalité clé est exprimée sous forme de user story avec des critères testables. Les fiches complètes se trouvent dans le dépôt, dans docs/user-stories/.

Réf.	User story	Critère d'acceptation déterminant
US-01	Parcourir et personnaliser les plats	Les options modifient le prix affiché en direct ; une option inconnue envoyée au serveur est ignorée
US-02	Suivre ma commande en temps réel	Le statut change en moins de 5 secondes, sans action de l'utilisateur
US-03	Payer ma commande en ligne	Aucune commande n'est enregistrée si le paiement échoue ; le panier est conservé
US-04	Noter le restaurant après livraison	La notation n'est possible qu'après livraison, et une seule fois par commande
US-05	Recevoir les commandes (restaurant)	Je ne vois que les commandes de mon établissement ; sinon 403
US-06	Accepter une livraison (livreur)	Deux livreurs simultanés : un seul obtient la course, l'autre reçoit 409
US-07	Discuter au sujet d'une commande	Seules les parties prenantes accèdent à la discussion

6. Contraintes techniques

La pile ci-dessous est celle du produit livré. Les écarts par rapport à la version 2.0 sont signalés et justifiés en section 16.

Couche	Technologie retenue	Rôle
Frontend mobile	Expo SDK 54, React Native 0.81, TypeScript, expo-router	Interface des quatre rôles (Android, iOS, web)
Gestion d'état	Context API (React)	Session et panier ; Redux jugé disproportionné
Backend	Node.js 20+, Express 4	API REST, contrôle d'accès, logique métier
Base de données	MongoDB 6+, Mongoose 8	Sept collections, index métier
Temps réel	Socket.IO 4	Suivi de commande et discussion, authentifiés par JWT
Authentification	JWT (24 h) + bcrypt (12 tours)	Sessions et hachage des mots de passe
Session mobile	expo-secure-store	Jeton conservé dans le trousseau du système
Paieement	Stripe (mode test) ou simulation locale	Encaissement, sans manipulation de carte réelle
Cartographie	react-native-maps + expo-location	Position du livreur
Notifications	expo-notifications + service Expo Push	Alerte à chaque étape, avec repli local
Hébergement	Render ou Railway + MongoDB Atlas	Déploiement du backend
Tests	node:test, tsc --noEmit	30 tests unitaires, vérification des types
Intégration continue	GitHub Actions	Tests et types à chaque Pull Request
Outils	VS Code, Git, GitHub	Développement et contrôle de version

Note

Le paiement comptant a été retiré de la liste des intégrations logicielles : il s'agit d'une modalité de règlement, pas d'une intégration. Il reste offert au client sous le nom « paiement à la livraison ».

7. Exigences non fonctionnelles

Ces exigences sont chiffrées, donc vérifiables. La colonne « État » indique honnêtement ce qui a été mesuré et ce qui ne l'a pas été.

Catégorie	Exigence mesurable	État
Performance	Temps de réponse des API inférieur à 500 ms en conditions de test	Respecté (mesures manuelles, réseau local)

Catégorie	Exigence mesurable	État
Performance	Démarrage à froid de l'application inférieur à 3 secondes	Respecté en build de production
Temps réel	Changement de statut propagé en moins de 5 secondes	Respecté (constaté à moins d'une seconde en réseau local)
Charge	50 utilisateurs simultanés sans dégradation notable	Non mesuré — aucun test de charge réalisé
Ergonomie	Parcours de commande en 5 écrans ou moins	Respecté : catalogue, fiche, panier, paiement, suivi
Sécurité	Mots de passe hachés ; aucune donnée de carte stockée	Respecté
Sécurité	Limitation des tentatives de connexion	Respecté : 10 tentatives par 15 minutes et par adresse IP
Compatibilité	Android et iOS depuis une seule base de code	Respecté (web également fonctionnel)
Disponibilité	Backend accessible pendant la démonstration finale	Configurations Render et Railway versionnées, prêtes à déployer

8. Utilisateurs ciblés

Profil	Description	Besoins couverts
Clients	Étudiants, travailleurs, familles ayant peu de temps pour cuisiner	Commander, personnaliser, payer, suivre, discuter, noter
Restaurants partenaires	Restaurants locaux, fast-foods, pizzerias, cafés	Recevoir les commandes, faire évoluer leur statut
Livreurs	Livreurs indépendants ou salariés du partenaire	Voir les courses disponibles, en accepter une, la mener à terme
Administrateurs	Équipe d'exploitation de la plateforme	Superviser l'activité, gérer comptes, restaurants et commandes

Contrairement à la version 2.0, les livreurs et les administrateurs ne sont plus « décrits pour mémoire » : ils disposent d'un espace fonctionnel dans l'application.

9. Conformité et protection des données

L'application collecte des données personnelles (compte, adresses, téléphone), de la géolocalisation côté livreur et des informations liées au paiement. Même dans un cadre étudiant, la conception tient compte du contexte légal québécois.

Loi 25 (Québec)

Principe	Mise en œuvre dans le projet
Consentement	L'utilisateur consent à la collecte lors de l'inscription ; la géolocalisation fait l'objet d'une demande d'autorisation explicite du système
Minimisation	Seules les données nécessaires au service sont collectées. Le rôle ne peut pas être choisi à l'inscription
Sécurité	Mots de passe hachés (bcrypt, 12 tours), jetons d'accès à durée limitée, en-têtes de sécurité HTTP, CORS restreint en production
Exactitude	L'utilisateur peut corriger son profil à tout moment
Responsable de la protection	Rôle identifié au sein de l'équipe (coordination QA)
Droit à l'effacement	Suppression de compte disponible côté administration ; l'auto-suppression par l'utilisateur reste à implémenter

Paieement et PCI-DSS

En mode Stripe, le numéro saisi par l'utilisateur ne quitte jamais le serveur pour être transmis à Stripe : il sert uniquement à sélectionner un moyen de paiement de test. En mode simulation, seul le format est vérifié. Dans les deux cas, aucun numéro de carte n'est enregistré en base ; seules la référence de transaction et la date de paiement le sont.

Limite assumée

Une application de production ferait saisir la carte dans un composant Stripe certifié, de sorte que le numéro ne transite jamais par notre serveur. Ce n'est pas le cas ici, faute de compatibilité du SDK natif avec Expo Go. C'est acceptable parce qu'aucune carte réelle n'est utilisable : le serveur refuse de démarrer avec une clé Stripe de production.

10. Analyse de risques

Bilan des risques identifiés au cadrage, confronté à ce qui s'est réellement produit.

Risque	Prob.	Impact	Mitigation prévue	Ce qui s'est produit
Courbe d'apprentissage de React Native	Élevée	Moyen	Prototype en semaine 1, entraide	Survenu, amplifié par le passage à TypeScript ; absorbé
Dépendance aux API tierces	Moyenne	Élevé	Mode sandbox, données simulées	Survenu sur Stripe ; résolu par le double mode (ADR 0004)

Risque	Prob.	Impact	Mitigation prévue	Ce qui s'est produit
Retard de développement	Moyenne	Élevé	MVP priorisé, marge tampon	Survenu : la notation n'a été livrée qu'en fin de projet
Intégration temps réel complexe	Moyenne	Moyen	Tester tôt sur un cas simple	Maîtrisé : salons Socket.IO en place dès le Sprint 2
Dérive documentation / code	—	Élevé	Non identifié au cadrage	Survenu : /docs décrivait encore le Sprint 1. Repris intégralement
Indisponibilité d'un équipier	Faible	Élevé	Documentation à jour, revue croisée	Non survenu

11. Stratégie de test

Les tests accompagnent chaque tâche : une tâche n'est terminée que si le code, les tests et la documentation sont à jour dans la même Pull Request.

Niveau	Outil	Couverture	Automatisé
Unitaire	node:test (module natif)	Tarification, statuts, sécurité, erreurs asynchrones, validation du menu, notifications — 30 tests	Oui
Statique	tsc --noEmit	Cohérence des types entre l'API et les écrans	Oui
Intégration API	Postman / curl	Enchaînement des routes, codes de statut, autorisations	Non
Acceptation	Parcours sur téléphone	Scénario complet multi-rôles, cas d'erreur	Non

Cas d'erreur systématiquement rejoués avant chaque démonstration : double acceptation d'une livraison (409), accès croisé entre restaurants (403), double notation (409), notation avant livraison (409), carte refusée (402), dépassement du nombre de tentatives de connexion (429), recherche contenant une expression régulière malveillante.

12. Matrice de traçabilité

Chaque besoin est relié à une fonctionnalité, à une user story, à un test et à un état.

Besoin	Fonctionnalité	US	Test associé	État
Parcourir et personnaliser	Menu interactif	US-01	tarification.test.js + parcours manuel	Livré

Besoin	Fonctionnalité	US	Test associé	État
Suivre la commande	Suivi temps réel	US-02	statutsCommande.test.js + scénario multi-appareils	Livré
Payer en ligne	Paieement Stripe / simulation	US-03	securite.test.js (validation carte) + cartes de test	Livré
Évaluer l'expérience	Notation des restaurants	US-04	Parcours manuel + contrainte d'unicité en base	Livré
Traiter les commandes	Espace restaurant	US-05	statutsCommande.test.js + test d'accès croisé	Livré
Livrer les commandes	Espace livreur	US-06	Test de concurrence sur l'acceptation	Livré
Échanger sur une commande	Discussion	US-07	Parcours manuel multi-rôles	Livré
Superviser la plateforme	Espace administrateur	US-08	Test d'accès non autorisé (403)	Livré

13. Livrables

- Code source organisé : application Expo / React Native, backend Node.js, scripts d'administration.
- Application mobile fonctionnelle couvrant les quatre fonctionnalités imposées et les quatre rôles.
- Dépôt Git avec un historique lisible : un commit par fonctionnalité, branches main et develop.
- Documentation technique en Markdown dans /docs : architecture, base de données, API, installation, stratégie de test, décisions d'architecture (5 ADR).
- Suite de tests unitaires automatisée et intégration continue GitHub Actions.
- Cahier des charges v3.1 (le présent document) et document de conception.
- Documentation finale de présentation.
- Rapport de projet : objectifs, méthodologie, difficultés, solutions, résultats.
- Présentation et démonstration finales.

14. Calendrier réalisé

Semaine	Activité	Documentation produite
1	Analyse des besoins, dépôt, conventions d'équipe	README, structure /docs, ADR 0001 et 0002
2	Architecture et schéma MongoDB, prototype	architecture.md, base-de-donnees.md

Semaine	Activité	Documentation produite
	mobile	
3	Maquettes et user stories	Maquettes, user-stories/
4-5	Backend : API REST et authentification	docs/api/, document de conception Sprint 1
6-8	Écrans mobiles, menu, panier ; renommage en Savora	Notes d'étape, ADR 0003
9	Espaces restaurant et livreur, temps réel	Notes d'étape, ADR 0005
10	Paieement, discussion, espace administrateur	Notes d'étape, ADR 0004
11	Notation, sécurité, tests, gestion du catalogue, notifications, déploiement	Reprise complète de /docs, cahier des charges v3.1, 6 ADR
12	Présentation et démonstration finales	Documentation finale de présentation

Écart notable : la notation, prévue au Sprint 3 dès le cadrage, a été livrée en semaine 11. Le temps consacré aux espaces restaurant, livreur et administration l'avait repoussée. Ce report a été identifié lors de la revue de mi-sprint et rattrapé.

15. Critères de réussite

Critère	État
Les quatre fonctionnalités du MVP sont opérationnelles	Atteint
Le parcours complet fonctionne de bout en bout sur l'application mobile	Atteint, avec quatre rôles
Les exigences non fonctionnelles chiffrées sont respectées	Atteint, sauf le test de charge (non mesuré)
La documentation est complète, à jour et versionnée	Atteint après la reprise de la semaine 11
Les délais et livrables prévus sont respectés	Atteint, avec un report interne de la notation
La démonstration finale est validée par les encadrants	À évaluer

16. Écarts assumés par rapport à la version 2.0

Un cahier des charges qui prétendrait avoir tout prévu serait suspect. Voici les écarts, leur cause et leur justification.

Écart	Cause	Justification	Trace
Quatre rôles au lieu du seul client	Le suivi temps réel imposé n'avait aucun acteur pour déclencher « en route » et « livrée »	Sans espace livreur, la fonctionnalité 2 n'était démontrable qu'en simulant des changements de statut	ADR 0005
TypeScript au lieu de JavaScript	Erreurs répétées d'accès aux objets imbriqués renvoyés par l'API	Les types détectent à la compilation ce qui n'apparaissait qu'au test manuel	ADR 0003
Expo Router au lieu de react-navigation	Quinze écrans répartis sur quatre espaces	Le routage par fichiers supprime une configuration longue et redondante	ADR 0003
Context API sans Redux	L'état partagé se limite à la session et au panier	Redux aurait ajouté de la cérémonie sans bénéfice mesurable	ADR 0003
Stripe côté serveur, avec repli	Le SDK Stripe natif ne fonctionne pas dans Expo Go ; réseau non garanti en présentation	Le paiement reste démontrable en toutes circonstances, sans manipuler de carte réelle	ADR 0004
Pas de clé Google Maps	Coût et configuration ; le fournisseur natif suffit	L'écran de carte affiche la position réelle du livreur sans dépendance payante	architecture.md
Discussion par commande ajoutée	Besoin apparu au test : « où est mon livreur ? »	Le canal Socket.IO existait déjà ; coût marginal faible	ADR 0005
Notifications distantes indémontrables dans Expo Go	Depuis le SDK 53, Expo Go ne reçoit plus les push sur Android	Double canal : les vraies push sont implémentées, une notification locale prend le relais en démonstration	ADR 0006

17. Dette technique et perspectives

Dette technique identifiée

Élément	Conséquence	Correction envisagée
Aucun test d'intégration automatisé sur les routes	Les régressions d'autorisation ne sont détectées qu'en test manuel	Supertest + MongoDB en mémoire
Aucun test de charge	L'exigence « 50 utilisateurs simultanés » n'est pas vérifiée	Campagne k6 ou Artillery
Limitation de débit en mémoire de processus	Inefficace derrière plusieurs instances du serveur	Compteur partagé via Redis
Pas d'envoi de courriel	Le jeton de réinitialisation est renvoyé dans la réponse en développement	Service transactionnel (Resend, SendGrid)

Élément	Conséquence	Correction envisagée
Positions fixes du restaurant et du client sur la carte	Le trajet affiché n'est pas fidèle	Géocodage des adresses, position du livreur diffusée par Socket.IO
Notifications distantes non démontrables	Seul le canal local est visible en présentation	Development build EAS
Libellés de notification dupliqués	Serveur et application peuvent diverger sans que rien ne le signale	Source unique exposée par l'API

Perspectives fonctionnelles

- Pourboires et modération des avis.
- Connexion Google et authentification à deux facteurs pour les comptes d'administration.
- Auto-suppression du compte par l'utilisateur, exigée par le droit à l'effacement.

18. Glossaire

Terme	Définition
ADR	Architecture Decision Record : note documentant une décision technique, son contexte et ses conséquences
API REST	Interface permettant à l'application mobile de communiquer avec le serveur par HTTP
bcrypt	Algorithme de hachage des mots de passe, volontairement lent pour décourager la force brute
Index unique	Contrainte de base de données interdisant deux enregistrements de même valeur — ici, deux avis sur une même commande
JWT	JSON Web Token : jeton d'authentification signé, transmis à chaque requête
Loi 25	Loi québécoise sur la protection des renseignements personnels
MoSCoW	Méthode de priorisation : Must, Should, Could, Won't have
MVP	Produit minimum viable : version essentielle livrant la valeur principale
NoSQL	Base de données orientée documents, comme MongoDB
Opération atomique	Opération indivisible : deux requêtes simultanées ne peuvent pas produire un résultat incohérent
PaymentIntent	Objet Stripe représentant une intention de paiement et son cycle de vie
PCI-DSS	Norme de sécurité applicable au traitement des données de cartes bancaires
ReDoS	Déni de service par expression régulière : une saisie construite pour rendre une recherche extrêmement lente

Terme	Définition
Sandbox	Environnement de test isolé — ici, paiements Stripe sans argent réel
Socket.IO	Bibliothèque de communication bidirectionnelle en temps réel entre client et serveur
TypeScript	Sur-couche de JavaScript ajoutant un système de types vérifié à la compilation

19. Annexes

Les diagrammes de conception sont versionnés au format SVG dans le dépôt, sous docs/diagrammes/, afin de rester modifiables et de suivre l'évolution du code.

Annexe A — Architecture du système

Figure 1 — Architecture client-serveur : docs/diagrammes/architecture.svg

Annexe B — Schéma de la base de données

Figure 2 — Collections MongoDB et leurs relations : docs/diagrammes/schema_mongodb.svg

Annexe C — Parcours d'une commande

Figure 3 — Diagramme de séquence, du panier au paiement puis au suivi et à la notation : docs/diagrammes/sequence_commande.svg

Annexe D — Documentation technique du dépôt

- docs/architecture.md — vue d'ensemble, sécurité, limites connues
- docs/base-de-donnees.md — schémas, index, machine à états
- docs/api/README.md — routes REST et événements Socket.IO
- docs/installation.md — installation, comptes de démonstration, dépannage
- docs/tests/strategie-de-test.md — niveaux de test et couverture
- docs/decisions/ — les cinq ADR du projet